# Node.js’e Giriş

> **Kaynak:** Node.js resmî eğitim rehberi — Introduction to Node.js
> **Hazırlanma tarihi:** 16 Ekim 2026
> **Metin türü:** Resmî kaynağa dayanarak hazırlanmış özgün Türkçe teknik anlatım. Birebir çeviri değildir.
> **Kapsam:** İlk giriş makalesi, HTTP örneği ve ilgili açıklamaları.

## 1. Node.js nedir?

**Node.js**, tarayıcı dışında JavaScript çalıştırabilen açık kaynaklı, platformlar arası bir **JavaScript çalışma zamanı ortamıdır (runtime environment)**. Bu sayede JavaScript, yalnızca web sayfalarındaki etkileşimlerde değil; HTTP sunucuları, API hizmetleri, komut satırı araçları ve dosya işlemleri gibi alanlarda da kullanılabilir.

- **Açık kaynak (open-source):** Kaynak kodu incelenebilir ve geliştirilmesine katkıda bulunulabilir.
- **Platformlar arası (cross-platform):** Windows, macOS ve Linux üzerinde çalışır.
- **Çalışma zamanı (runtime):** JavaScript motoruyla birlikte işletim sistemiyle etkileşim sağlayan API’ler sunar.

Node.js bir programlama dili değildir: JavaScript dilinin kullanılabildiği bir çalışma ortamıdır.

## 2. V8 JavaScript motoru

Node.js, Google Chrome tarafından da kullanılan **V8 JavaScript motorunu** tarayıcı dışında çalıştırır. V8, JavaScript kodunu işleyerek yüksek performansla yürütülmesini sağlar. Bununla birlikte Chrome ve Node.js’in sunduğu ortam API’leri aynı değildir: Örneğin tarayıcıdaki `document` nesnesi Node.js ortamında varsayılan olarak bulunmaz.

## 3. Tek süreç, eş zamanlı bağlantılar ve engellemeyen G/Ç

Node.js uygulaması bir **süreç (process)** içinde çalışabilir; gelen her ağ isteği için ayrı bir işletim sistemi iş parçacığı açması gerekmez. Node.js standart kütüphanesindeki birçok API, **asenkron giriş/çıkış (I/O)** modelinden yararlanır. Böylece ağ, dosya veya veritabanı yanıtı beklenirken JavaScript yürütme akışı başka işleri ele alabilir.

**Engelleyen (blocking)** çağrı, yürütmeyi işlem sonuçlanana kadar durdurur. **Engellemeyen (non-blocking)** yaklaşım ise işlem sürerken kontrolün başka işlere geçmesine olanak tanır. Sonuç hazır olduğunda callback, Promise veya benzeri mekanizmalarla işlenir.

Bu tasarım, tek bir sunucunun binlerce eş zamanlı bağlantıyı yönetebilmesini kolaylaştırır. Fakat her türlü işlemin otomatik olarak paralel çalıştığı anlamına gelmez. Özellikle yoğun CPU hesaplamaları ana JavaScript iş parçacığını meşgul edebilir; Node.js ayrıca belirli görevler için işletim sistemi olanaklarından veya libuv iş parçacığı havuzundan yararlanır.

## 4. İstemci ve sunucu tarafında JavaScript

Node.js'in önemli avantajlarından biri, JavaScript bilen geliştiricilerin aynı dili hem web arayüzünde hem sunucu tarafında kullanabilmesidir. Örneğin bir tarayıcı istemcisi Node.js üzerinden geliştirilen HTTP API'sine istek gönderebilir; API de gerektiğinde bir veritabanıyla görüşerek yanıt döndürebilir.

Node.js kendi başına bir veritabanı değildir. PostgreSQL, MySQL ve MongoDB gibi sistemlerle uygun paketler aracılığıyla bağlantı kurulabilir.

## 5. ECMAScript desteği

**ECMAScript**, JavaScript'in standartlaştırılmış dil özelliklerini tanımlar. Tarayıcı uygulamalarında kullanıcıların tarayıcı sürümleri farklı olabilir; sunucu tarafında ise çalıştırdığınız Node.js sürümünü siz seçerek desteklenen dil özelliklerini daha doğrudan kontrol edebilirsiniz.

Bazı yeni veya deneysel özellikler belirli Node.js sürümlerinde komut satırı **bayrakları (flags)** ile etkinleştirilebilir. Hangi özelliğin kullanılabildiğini sürüme göre doğrulamak gerekir.

## 6. İlk HTTP uygulaması — Hello World

Node.js'in yerleşik `node:http` modülüyle basit bir HTTP sunucusu kurulabilir. Kaynak makalede aynı örnek iki modül sözdizimiyle verilir: **CommonJS (CJS)** ve **ECMAScript Modules (ESM)**. İkisi de aynı HTTP sunucusunu oluşturur.

### 6.1. CommonJS (CJS)

Dosya: `server.cjs`

```
const { createServer } = require('node:http');

const hostname = '127.0.0.1';
const port = 3000;

const server = createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

## 6.2. ECMAScript Modules (ESM)

Dosya: `server.mjs`

```
import { createServer } from 'node:http';

const hostname = '127.0.0.1';
const port = 3000;

const server = createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

**Modül seçimi:** `require()` CommonJS sisteminde, `import` ise ESM sisteminde kullanılır. `.cjs` ve `.mjs` uzantıları örneğin modül türünü açıkça belirtir. `.js` dosyalarında yorumlama proje yapılandırmasına bağlı olabilir.

## 6.3. Kodun satır satır açıklaması

| Kod | İşlevi |
|---|---|
| `require('node:http')` / `import ... from 'node:http'` | Yerleşik HTTP modülündeki `createServer` fonksiyonuna erişir. |
| `hostname = '127.0.0.1'` | Sunucuyu yerel bilgisayarın döngü adresine bağlar. |
| `port = 3000` | Dinlenecek TCP portunu belirler. |

| Kod | İşlevi |
| --- | --- |
| `createServer((req, res) => {...})` | Her HTTP isteği için çalışacak işlemciyi tanımlar ve sunucu nesnesini döndürür. |
| `req` | `http.IncomingMessage`: HTTP isteğinin ayrıntılarına erişimi sağlar. |
| `res` | `http.ServerResponse`: HTTP yanıtının gönderilmesini sağlar. |
| `res.statusCode = 200` | Başarılı yanıtı ifade eden HTTP durum kodunu ayarlar. |
| `res.setHeader('Content-Type', 'text/plain')` | İçeriğin düz metin olduğunu belirten yanıt başlığını ayarlar. |
| `res.end('Hello World')` | Metni göndererek yanıtı bitirir. |
| `server.listen(...)` | Sunucuyu belirtilen adres ve portta dinlemeye başlatır. |

`createServer()` yeni bir HTTP sunucusu döndürür. Sunucu istek aldığında bir **`request` olayı** oluşur; örnek, bunun işleyicisi olan fonksiyona `req` ve `res` nesnelerini verir. Bu basit örnekte `req` kullanılmasa da istek başlıklarına ve gövdesine ulaşmak mümkündür.

## 6.4. Uygulamayı çalıştırma

1. Bilgisayarında Node.js kurulu olduğundan emin ol (`node --version`).
2. İki kod örneğinden birini belirtilen dosya adıyla kaydet.
3. Terminali dosyanın bulunduğu klasörde aç.
4. Seçtiğin örneğe göre komutu çalıştır:

```
node server.mjs
# veya
node server.cjs
```

Terminalde şu mesaj görünür:

```
Server running at http://127.0.0.1:3000/
```

Ardından tarayıcıda **http://127.0.0.1:3000/** adresini aç. Yanıt olarak **Hello World** görürsün. Sunucuyu sonlandırmak için terminalde `Ctrl+C` kullanabilirsin.

> **Güvenlik ve ağ notu:** `127.0.0.1` yalnızca yerel bilgisayardan erişim için uygundur. Bu örnek, kimlik doğrulamalı veya internete açık üretim sunucusu değildir.

## 7. Teknik terimler

| İngilizce | Türkçe |
|---|---|
| Runtime Environment | Çalışma zamanı ortamı |
| JavaScript Engine | JavaScript motoru |
| Cross-platform | Platformlar arası |
| Process | Süreç |
| Thread | İş parçacığı |
| Asynchronous I/O | Asenkron giriş/çıkış |
| Blocking / Non-blocking | Engelleyen / Engellemeyen |
| Concurrent Connections | Eş zamanlı bağlantılar |
| HTTP Request / Response | HTTP isteği / yanıtı |
| Callback | Geri çağırma fonksiyonu |
| Standard Library | Standart kütüphane |

## 8. Kaynaklar ve kapsam notları

- Introduction to Node.js — resmî kaynak
- Node.js Learn — tüm öğrenme bölümleri
- Node.js HTTP API referansı

Bu metin, ilk resmî Node.js giriş makalesinin tüm ana konu alanlarını ve iki HTTP kod örneğini kapsayan **özgün teknik uyarlamadır**; resmî metnin kelimesi kelimesine çevirisi değildir. İşlenen ana kavramlar, daha kolay öğrenilebilmesi amacıyla genişletilmiş ve ek açıklamalarla desteklenmiştir.